

wavewatch lab — III & IV

A self-paced course on WAVEWATCH III, from the wave spectrum to GPUs,
WW4 and SWAN

Matheus Hoffmann

2026-09

Contents

1	00 — Orientation: what WW3 actually computes	1
1.1	The one equation	1
1.2	The source terms	1
1.3	Phase-averaged, not phase-resolving	2
1.4	The five dimensions	2
1.5	The program pipeline	2
1.6	Two input file formats	3
1.7	Where to go next	3
2	01 — Getting it built	4
2.1	Two halves	4
2.2	Dependencies	4
2.3	The switch file	4
2.4	Build	5
2.5	Prove it works	5
2.6	Things that go wrong	5
2.7	Building with nvfortran	6
3	02 — Anatomy of a run	7
3.1	ww3_grid — the most important program	7
3.2	ww3_strt — initial conditions	9
3.3	ww3_prnc — forcing	9
3.4	ww3_shel — the model	9
3.5	ww3_ounf / ww3_ounp — output	9
4	03 — Grids, bathymetry, masks	11
4.1	Four grid types	11
4.2	Grid closure	11
4.3	Bathymetry	11

4.4	Masks	12
4.5	Obstruction grids — the underrated one	12
4.6	Choosing resolution	12
4.7	Exercise	13
5	04 — Forcing: winds, currents, ice, water levels	14
5.1	What WW3 will accept	14
5.2	Winds are everything	14
5.3	ww3_prnc	15
5.3.1	The four things that go wrong	15
5.3.2	Verify before you run the model	16
5.4	Boundary spectra	16
5.5	Exercise	16
6	05 — Nesting and multi-grid	17
6.1	One-way nesting: chained ww3_shel runs	17
6.2	Two-way: ww3_multi	18
6.3	Which to use	18
6.4	Unstructured as an alternative	18
7	06 — Output and post-processing	19
7.1	The seven output types	19
7.2	Fields worth knowing	19
7.3	Partitioning: the thing to actually understand	20
7.4	Reading the output	20
7.5	Validation	21
8	07 — Physics choices: which source terms, and why it matters	22
8.1	The source-term packages	22
8.2	Why this matters more than you'd like	22
8.3	The nonlinear term	23
8.4	Propagation	23
8.5	Shallow water	23
8.6	Sea ice	24
8.7	Tuning namelists	24
8.8	A note on tuning	24

9 08 — Driving WW3 from Python	25
9.1 The landscape	25
9.2 pyww3	25
9.2.1 Minimal end-to-end	26
9.2.2 The honest assessment	26
9.2.3 If pyww3 doesn't fit	26
9.3 Work through the exercises	27
1009 — Performance, and the RTX 4090 question	28
10.1 Your question, answered directly	28
10.2 Why WW3 on a GPU is hard	28
10.2.1 There is no GPU support upstream	28
10.2.2 The one serious attempt got 1.3×	29
10.2.3 And WW3 is not where the GPU work is going	29
10.2.4 Your hardware makes it worse, not better	29
10.2.5 The structural problem	30
10.3 What to do instead (in order)	30
10.3.11. Use the parallelism WW3 already has	30
10.3.22. Build WW3 with nvfortran — on the CPU	30
10.3.33. Measure your own hardware first	31
10.3.44. Profile before you port anything	31
10.3.55. If you still want to port — the realistic plan	31
10.3.66. Point the 4090 at something it's actually good at	32
10.4 Quick reference: nvfortran	32
10.5 Realistic performance expectations, summarised	33
10.6 Sources for everything above	33
1110 — WAVEWATCH IV: what's coming, and how mature it is	34
11.1 It exists	34
11.2 Maturity: a snapshot	34
11.3 Why a new model rather than WW3 v8	35
11.4 Languages — the contentious part	35
11.5 Yes, it is being written partly by AI agents	36
11.6 Timeline	36
11.7 What this means for WW3 — and for you	37
11.8 One thing that must survive	37

1211 — SWAN: the other one you should know	38
12.1 What it is	38
12.2 Where the source actually lives	38
12.3 Building it	39
12.3.1 Switches, SWAN-style	39
12.4 SWAN versus WW3: the actual difference	39
12.5 When to use which	40
12.6 The ecosystem around it	40
12.7 Exercise	40

Chapter 1

00 — Orientation: what WW3 actually computes

1.1 The one equation

WW3 solves the **spectral action density balance equation**. At every point in space and every moment in time, it carries a two-dimensional spectrum: how much wave energy sits at each frequency and each direction.

$$\frac{\partial N}{\partial t} + \nabla_{\mathbf{x}} \cdot (\dot{\mathbf{x}} N) + \frac{\partial}{\partial k} (\dot{k} N) + \frac{\partial}{\partial \theta} (\dot{\theta} N) = \frac{S}{\sigma}$$

- $\frac{\partial}{\partial k} (\dot{k} N) + \frac{\partial}{\partial \theta} (\dot{\theta} N) = \frac{S}{\sigma}$

where $N(k, \theta; \mathbf{x}, t) = F/\sigma$ is **action** density (energy over intrinsic frequency — conserved in the presence of currents, where energy isn't).

Read the terms left to right and you have the whole model:

Term	Physics	WW3 switch/flag
$\partial N / \partial t$	evolution in time	—
$\nabla_x \cdot (\dot{x} N)$	waves travel at the group velocity	RUN%FLCX, RUN%FLCY
$\partial(\dot{k} N) / \partial k$	shoaling, current-induced frequency shift	RUN%FLCK
$\partial(\dot{\theta} N) / \partial \theta$	refraction by depth and current	RUN%FLCTH
S / σ	all the physics you can't derive	RUN%FLSOU

The left side is clean conservative transport in a 4D phase space (x, y, k, θ). It's numerics, and it's solvable. The right side is where the model earns its keep and where all the uncertainty lives.

1.2 The source terms

$$S = S_{in} + S_{nl} + S_{ds} + S_{bot} + S_{db} + S_{ice} + \dots$$

- S_{in} — **wind input**. Momentum from the atmosphere into the waves. The parameterisation you pick (ST2, ST4, ST6) mostly means picking a version of this.
- S_{nl} — **nonlinear wave-wave interaction**. Four-wave resonant quartets shuffle energy between frequencies. This is the term that makes the spectrum evolve towards its characteristic shape rather than just growing where the wind pushes. The exact computation is $\sim 1000\times$ the cost of everything else combined, so everybody uses the Discrete Interaction Approximation (NL1, Hasselmann 1985) and everybody knows it's the weakest link.
- S_{ds} — **dissipation**, mostly whitecapping. Historically the tuning term: whatever was needed to make $S_{in} + S_{nl}$ balance out to observed spectra.
- S_{bot} , S_{db} — bottom friction, depth-induced breaking. Shallow water only.
- S_{ice} — ice attenuation and scattering (IC1-IC5, IS1/IS2).

The crucial thing to internalise: S_{in} and S_{ds} are individually large and individually uncertain, and they nearly cancel. The net is a small difference of big numbers. That's why source-term packages come as matched sets — you cannot mix the ST4 input with the ST6 dissipation and expect anything sensible.

1.3 Phase-averaged, not phase-resolving

WW3 does **not** know where any individual wave crest is. It tracks statistics. If you want a surface you can render, or wave-by-wave runup on a beach, or slamming loads on a hull, you want a different class of model entirely (SWASH, XBeach, FUNWAVE, OpenFOAM — see docs/AWESOME-WW3_202609.md).

What WW3 gives you is: the spectrum, and everything derivable from it. H_s , peak period, mean direction, directional spread, Stokes drift, radiation stresses, partitioned swell systems.

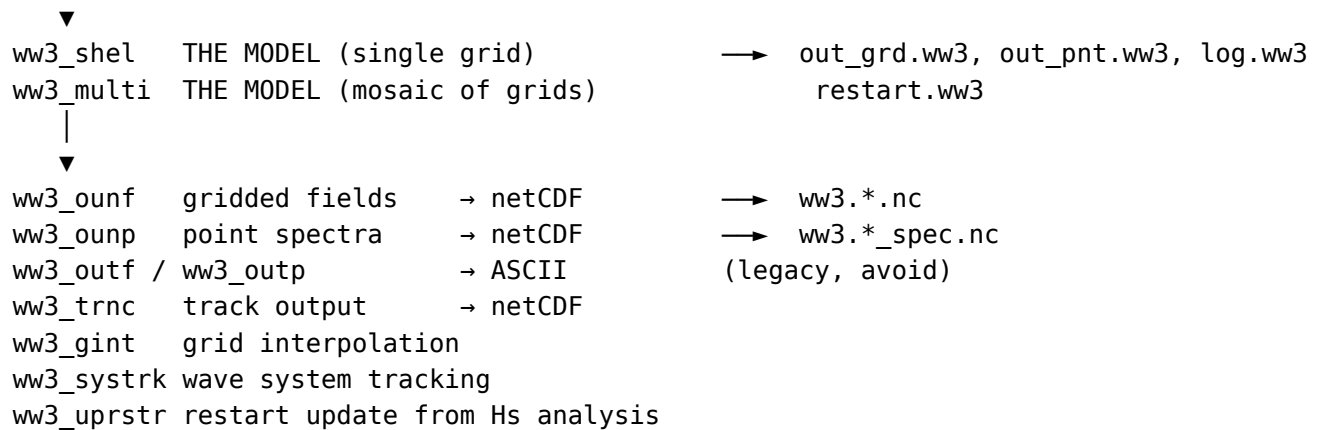
1.4 The five dimensions

Every field WW3 carries lives on **latitude $\times$ longitude $\times$ frequency $\times$ direction $\times$ time**. A modest regional run — 81×81 points, 32 frequencies, 36 directions — is 7.5 million numbers *per timestep*, before any output. This is why WW3 is expensive, and it is directly relevant to the GPU question in lesson 09: you cannot casually shuttle that array across PCIe every step.

1.5 The program pipeline

WW3 isn't one binary. It's a family, communicating through files:

ww3_grid	bathymetry + spectrum + timesteps	→	mod_def.ww3	[THE model definition]
ww3_strt	initial conditions	→	restart.ww3	
ww3_prnc	netCDF forcing → binary	→	wind.ww3, current.ww3, ice.ww3	
ww3_bounc	spectra from a parent run	→	nest.ww3	



mod_def.ww3 is the centre of gravity. It's an opaque binary produced by `ww3_grid` containing the complete model definition — grid, spectral discretisation, timesteps, physics configuration. Every other program reads it. Change `ww3_grid.nml`, and every downstream artifact is stale until you rerun `ww3_grid`. A large share of all confusing WW3 behaviour reduces to a stale `mod_def`.

1.6 Two input file formats

WW3 accepts **legacy** `.inp` (fixed-format, positional, comment lines start with `$`) and **modern** `.nml` (Fortran namelists, order-independent, self-documenting). Both work in v7. Everything in this repo uses `.nml`, because:

- the annotated templates in `$WW3/model/nml/` document every single parameter inline;
- you can omit anything you want defaulted;
- `pyww3` and every other Python wrapper targets namelists.

If you follow an older tutorial and get error reading input file, check whether it's handing an `.inp` to a program expecting `.nml` or vice versa. That's a real and common failure mode.

1.7 Where to go next

→ [01-build.md](#) — get it compiled.

Chapter 2

01 — Getting it built

2.1 Two halves

The WW3 package is **the git repo plus a binary data bundle**. The repo has the Fortran and the regression-test *configurations*; the bundle has the bathymetry, forcing, and reference output the tests need. The bundle lives on NOAA's FTP, not in git.

```
git clone --branch develop https://github.com/NOAA-EMC/WW3.git ~/src/WW3
cd ~/src/WW3
./model/bin/ww3_from_ftp.sh      # the other half. Slow. Required.
export WW3=$PWD
```

Don't use the GitHub release tarball. The newest tagged release is 6.07.1, from April 2019. Real work happens on develop and main, and the wider community is running v7.14.x. Clone a branch.

2.2 Dependencies

- Fortran 90 compiler — gfortran, ifort/ixf, nvfortran, or Cray
- **CMake ≥ 3.19**
- **NetCDF ≥ 4.1.1 with Fortran bindings.** The C library alone won't do; ww3_ounf and friends need netcdf.mod. On Debian/Ubuntu: libnetcdf-dev.
- MPI for anything realistic

bash scripts/00_prereqs.sh installs all of it on Debian/Ubuntu.

2.3 The switch file

Before you build you pick a **switch file**: a whitespace-separated list of CPP keys that selects the physics and the parallelism at compile time. See [../switches/README.md](#) for a full annotated breakdown.

The consequence that matters: **changing the switch file requires a full rebuild.** Not make, a full `rm -rf build`. The switch is baked into the preprocessed sources. "I turned on ST4 and nothing changed" is nearly always a stale build directory.

2.4 Build

```
cd $WW3
rm -rf build && mkdir build && cd build
cmake .. -DSWITCH=/abs/path/to/ww3-lab/switches/switch_lab_shrd
make -j$(nproc)
ls bin/
```

-DSWITCH takes either a bare name resolved inside `$WW3/model/bin/`, or an absolute path to your own file.

Useful variants:

```
cmake --build . --target ww3_shel          # rebuild one program
cmake .. -DCMAKE_BUILD_TYPE=Debug -DSWITCH=... # bounds checking, backtraces
export NetCDF_ROOT=$(nf-config --prefix)    # if CMake can't find NetCDF
```

2.5 Prove it works

```
bash scripts/03_run_regtest.sh $WW3 ww3_tp2.2
```

That script runs each program in turn and tees its stdout, rather than hiding everything behind the `run_test` harness. Watch what each one prints. `ww3_grid` in particular emits a long, genuinely useful summary of the grid it built and every namelist it read — always keep it: `ww3_grid | tee ww3_grid.out`.

2.6 Things that go wrong

Symptom	Cause
Cannot open include file 'netcdf.inc' / undefined netCDF symbols	NetCDF Fortran bindings
<code>ww3_ounf</code> runs but writes no <code>.nc</code>	NC4 missing from the switch
<code>ww3_grid</code> demands an obstruction file you don't have	FLAGTR in <code>namelists.nml</code> is
Model runs, output is all zeros	Forcing never arrived. C
Changed physics, nothing changed	Stale <code>build/</code> . <code>rm -rf build</code>
Changed the grid, downstream programs behave oddly	Stale <code>mod_def.ww3</code> . Rerun
error reading input file	<code>.inp</code> vs <code>.nml</code> mismatch.

2.7 Building with nvfortran

Possible and worth doing, but you must rebuild HDF5 and NetCDF with nvfortran first — distro packages are gfortran-built and their `.mod` files are unusable. Details and a realistic assessment in [09-gpu-and-performance.md](#).

→ [02-anatomy-of-a-run.md](#)

Chapter 3

02 — Anatomy of a run

Work through [examples/01-fetch-limited-growth](#) with this open. Every claim here is visible in those files.

3.1 ww3_grid — the most important program

It reads `ww3_grid.nml` plus your ASCII bathymetry/mask, and writes `mod_def.ww3`. Its namelist blocks, in dependency order:

&SPECTRUM_NML — the spectral discretisation.

```
SPECTRUM%FREQ1 = 0.04118    ! lowest frequency, Hz
SPECTRUM%XFR    = 1.1       ! geometric ratio between bins
SPECTRUM%NK     = 32        ! number of frequencies
SPECTRUM%NTH    = 24        ! number of directions
```

$f_{\text{max}} = \text{FREQ1} \times \text{XFR}^{(\text{NK}-1)}$. With the defaults that's $0.04118 \rightarrow 0.76$ Hz, i.e. periods from 24 s down to 1.3 s. **Energy outside this range does not exist in your model.** If you care about 26-second Southern Ocean swell, lower FREQ1. Nothing warns you.

NTH = 24 means 15° bins. That's coarse for swell. Coarse direction resolution produces the **Garden Sprinkler Effect**: a smooth swell field propagating far from its source breaks up into discrete beams, because each direction bin travels as an independent ray. The PR3 propagation scheme includes a correction; more directions helps more.

&RUN_NML — switch individual terms of the balance equation on and off. Turning `FLSOU = F` gives pure propagation, which is how the `ww3_tp*` regression tests isolate numerics from physics. Extremely useful for debugging.

&TIMESTEPS_NML — four numbers, and they are not tuning knobs.

```
DTMAX    global timestep – everything else divides into it
DTXY     spatial propagation – CFL-limited
DTKTH    refraction / wavenumber shift
DTMIN    minimum source-term integration step
```

The CFL limit comes from the fastest group velocity, which belongs to the *lowest* frequency (deep water: $c_g = g/(4\pi f)$, so low $f \rightarrow$ fast):

```
cg_max = g / (4π·FREQ1)
T_cfl  = Δx_min / cg_max
DTXY   ≈ 0.9 · T_cfl      (round DOWN)
DTMAX  ≈ 3 · DTXY
DTKTH  ≈ DTMAX / 2        (÷10 if you have strong currents)
DTMIN  ≈ 10 s
```

Keep them integer multiples of each other. Round down, never up.

&GRID_NML — TYPE is RECT / CURV / UNST; COORD is SPHE / CART; CLOS is NONE / SMPL (periodic in i) / TRPL (tripole, for global grids that need to handle the North Pole). ZLIM is the depth above which a point is permanently land; DMIN is the floor applied to depth in the physics so shallow points don't blow up.

&DEPTH_NML / &MASK_NML / &OBST_NML — ASCII input arrays. Three fields to get right:

- SF — scale factor, **multiplied** into the values you supply. **Depths must be negative below mean sea level.** If your file has positive depths, set SF = -1.
- IDLA — layout. 1 = line by line from the *bottom* row; 3 = line by line from the *top*. Getting this wrong flips your continent upside down and is not always obvious.
- IDFM / FORMAT — free vs fixed format.

Mask legend: 0 land, 1 sea, 2 **active open boundary**, 3 excluded, 7 ice, -1/-2 ice-excluded.

&INBND_COUNT_NML / &INBND_POINT_NML — where boundary spectra enter. Two rules: boundary points must be strictly *inside* the grid (never the first or last row/column, the propagation stencil needs a cell outside them), and CONNECT = T auto-fills every point on the straight line from the previous entry.

GRID%NML = 'namelists.nml' — points at a second file holding the physics tuning namelists (&MISC, &SIN4, &SDS4, &PR03, &UNST, ...). Blocks not needed by your switch settings are skipped automatically. If you have no obstruction grid, &MISC FLAGTR = 0.

Run it, always keeping stdout:

```
ww3_grid | tee ww3_grid.out
```

It prints the whole grid summary, every namelist value it read, and the count of sea/land/ boundary points. That count is your first sanity check.

Outputs: mod_def.ww3 (binary, the model definition), mapsta.ww3 (status map), mask.ww3 (ASCII land/sea — open it and look at it).

3.2 ww3_strt — initial conditions

Writes `restart.ww3`. If it's absent, `ww3_shel` cold-starts from a calm sea and says so in the log, which is usually what you want for a spin-up run. For a specific initial spectrum (the propagation tests seed a Gaussian packet and watch it travel), copy an `ww3_strt` input from a regtest — `$WW3/regtests/ww3_tpl.1/input/` is the canonical one.

3.3 ww3_prnc — forcing

`netCDF` → `WW3` binary, interpolated onto your grid. One invocation per forcing type; you swap `ww3_prnc.nml` between runs. The failure modes are boring and universal: wrong variable names, wrong coordinate names, wrong longitude convention (0-360 vs -180-180), wrong time units. `ncdump -h` before you start.

3.4 ww3_shel — the model

Reads `ww3_shel.nml` and `mod_def.ww3`.

- **&DOMAIN_NML** — start and stop, 'YYYYMMDD HHMMSS'.
- **&INPUT_NML** — per forcing: 'T' read from file, 'H' homogeneous (given below), 'C' from a coupler. Boundary spectra are *not* flagged here — `nest.ww3` is picked up automatically if present.
- **&OUTPUT_TYPE_NML** — what to output. Seven output types exist: gridded fields, point spectra, track output, restarts, boundary data, separated wave fields, coupling fields.
- **&OUTPUT_DATE_NML** — when. Each entry is `START STRIDE_SECONDS STOP`. A stride of '0' disables that output type.
- **&HOMOG_COUNT_NML / &HOMOG_INPUT_NML** — constant forcing without any data files. For 'WND': speed, direction, air-sea ΔT .

Watch `log.ww3` while it runs. It prints a timestep table with a column per input showing which forcings updated when — X for an update, F for a file read. If a column never moves, your forcing isn't arriving.

For MPI builds: `mpirun -np N ww3_shel`.

3.5 ww3_ounf / ww3_ounp — output

`ww3_ounf` turns `out_grd.ww3` into gridded `netCDF`. `ww3_ounp` turns `out_pnt.ww3` into spectral `netCDF`.

`FIELD%TIMESPLIT` controls file chunking: 0 one file, 4 yearly, 6 monthly, 8 daily, 10 hourly. `FIELD%TYPE` is 2 (packed `SHORT`), 3 (mixed), 4 (`REAL`) — use 4 while learning so you never wonder whether an odd value is a packing artefact.

The field name list is long and documented at the top of `$WW3/model/nml/ww3_ounf.nml`. The ones you'll use constantly:

HS	significant wave height	T01/T02/T0M1	mean periods
FP	peak frequency	DIR SPR	mean direction / spread
WND	wind (echo it back, always)	DPT	depth
PHS PTP PDIR PSPPR	partitioned wind sea + swell systems		
EF	1D frequency spectrum	USS TUS	Stokes drift
SXY	radiation stresses	UST	friction velocity

Always output WND and DPT. They cost nothing and they catch forcing and grid errors immediately.

→ [03-grids.md](#)

Chapter 4

03 — Grids, bathymetry, masks

4.1 Four grid types

Type	GRID%TYPE	When
Rectilinear	RECT	Regular lat-lon or Cartesian. Default. Cheapest per point.
Curvilinear	CURV	Body-fitted logically-rectangular mesh; you supply x and y coordinates.
Unstructured	UNST	Triangular mesh from GMSH .msh. Resolution where you need it.
SMC	via &SMC_NML	Spherical Multiple-Cell: quad-tree refinement on a sphere. Used for global grids.

Unstructured is where coastal work is going, because the resolution mismatch between a 1000 km shelf and a 200 m inlet is brutal on a rectilinear grid. The cost: unstructured WW3 uses implicit schemes and needs PDLIB + ParMetis for domain decomposition (added v6.04), which is a build-time complication.

4.2 Grid closure

GRID%CLOSE matters only for global spherical grids:

- NONE — no wrap. Regional.
- SMPL — periodic in i ; $(NX+1, J) \rightarrow (1, J)$. Forces $SX = 360/NX$.
- TRPL — tripole. Periodic in i *and* folded at $j = NY+1$: $(I, NY+1) \rightarrow (NX-I+1, NY)$. Requires NX even and a curvilinear grid. This is how you avoid the North Pole singularity on a global grid.

4.3 Bathymetry

Sign convention: negative below mean sea level. This is opposite to how most people think about "depth" and it is the single most common grid error. GEBCO's elevation is already negative below sea level, so $DEPTH\%SF = 1$. For a file of positive depths, $DEPTH\%SF = -1$.

IDLA decides which way is up. 1 = bottom row first, 3 = top row first. Check by comparing `mask.wv3` (ASCII, written by `wv3_grid`) against a map. Do it visually, once, every time you build a new grid.

ZLIM vs DMIN. ZLIM (e.g. -0.10) is the coastline: points shallower than this are permanently excluded and will never be wet even if water level rises. DMIN (e.g. 2.50) is a floor applied to the depth used in the physics, so a nearly-dry cell doesn't produce division-by-zero. They do different things and both matter.

4.4 Masks

- 2 excluded boundary point (ice-covered)
- 1 excluded sea point (ice-covered)
- 0 excluded land point
- 1 sea point
- 2 active boundary point
- 3 excluded grid point
- 7 ice point

Note 2 — boundary points are declared in the *mask*, and/or via `&INBND_POINT_NML` in `wv3_grid.nml`. `wv3_grid` will promote active points on the declared boundary segments to status 2 and report how many.

4.5 Obstruction grids — the underrated one

At 0.1° you cannot resolve a 3 km island. But that island blocks waves. WW3's answer is **subgrid obstruction**: a per-cell transparency in x and y, between 0 (fully blocking) and 1 (fully transparent), applied during propagation.

Controlled by `&MISC FLAGTR` in `namelists.nml`:

- 0 no obstruction (and then you must NOT supply an obstruction file)
- 1 transparency at cell boundaries
- 2 transparency at cell centres
- 3 as 1, with continuous ice
- 4 as 2, with continuous ice

Generate these with `gridgen` (MATLAB). Without them, archipelagos are invisible and your lee-side wave heights are badly wrong. With them, a coarse global grid produces surprisingly decent results in the Indonesian seas and the Aegean.

4.6 Choosing resolution

Two competing constraints:

1. **Physics.** Resolve the bathymetric features that refract the waves you care about.

2. **CFL.** $\Delta t \propto \Delta x$. Halve the grid spacing and you double the timestep count *and* quadruple the point count — $8\times$ the cost for a 2D grid.

The standard resolution is a nested mosaic: a coarse global grid (0.5°) feeding a regional grid (0.1°) feeding a coastal grid (0.01°). Either as separate runs chained through `ww3_bounc`, or as a single `ww3_multi` run with two-way interaction. See lesson 05.

4.7 Exercise

Build the same domain three ways — 0.2° , 0.1° , 0.05° — with correctly recomputed timesteps, and run example 02's forcing through all three. Plot H_s at the shelf-break point. Where does it converge? Where doesn't it? That answer is specific to your coastline and it's the only honest way to pick a resolution.

→ [04-forcing.md](#)

Chapter 5

04 — Forcing: winds, currents, ice, water levels

5.1 What WW3 will accept

Forcing	Why it matters
10 m winds	The engine. Everything else is a correction.
Currents	Refraction, frequency shift, and the Agulhas/Gulf Stream giant-wave
Water levels	Tides. Matters in shallow water, irrelevant offshore.
Ice concentration	Attenuates and scatters.
Ice thickness/floe size	Needed by the more sophisticated ice physics (IC2-IC5).
Mud	Dissipation over muddy bottoms. Niche.
Air density, momentum	Mostly for coupled configurations.

Each takes 'T' (from a file), 'H' (homogeneous constant, given in &HOMOG_INPUT_NML), or 'C' (from a coupler).

5.2 Winds are everything

Wave model error is dominated by wind error, not by wave physics. A 10% wind speed error becomes roughly a 20% error in H_s in a growing sea (energy goes roughly as U^2 in the fetch-limited laws, and worse than that in the growth phase). Before you spend a week tuning BETAMAX, check your winds against scatterometer data.

Practical consequences:

- **Resolution.** ERA5 at 0.25° cannot represent a squall line or a tight tropical cyclone core. For storm cases you want a higher-resolution atmospheric model, not a finer wave grid.
- **Temporal frequency.** Hourly is the minimum for anything with a front in it. 6-hourly winds smear out exactly the events you care about.

- **Height.** WW3 wants 10 m winds. If your source gives a different level, convert with a log profile — or better, use &SIN4 ZWND / the equivalent for your source-term package to tell WW3 what height it's being given.
- **Stability.** HOMOG_INPUT(i)%VALUE3 and the STAB switch handle air-sea temperature difference. A cold-air outbreak over warm water generates markedly more wave energy than the same wind speed in stable conditions.

5.3 ww3_prnc

```
cp ww3_prnc_wind.nml ww3_prnc.nml
ww3_prnc | tee ww3_prnc_wind.out
```

It interpolates a netCDF field onto your model grid and writes WW3's binary format (wind.ww3, current.ww3, ice.ww3, level.ww3). Key namelist fields:

```
&FORCING_NML
  FORCING%FIELD%WINDS = T
  FORCING%GRID%LATLON = T
  FORCING%TIMESTART   = 'YYYYMMDD HHMMSS'
  FORCING%TIMESTOP    = 'YYYYMMDD HHMMSS'
/
&FILE_NML
  FILE%FILENAME = 'era5_winds.nc'
  FILE%LONGITUDE = 'longitude'
  FILE%LATITUDE = 'latitude'
  FILE%VAR(1)    = 'u10'
  FILE%VAR(2)    = 'v10'
/
```

5.3.1 The four things that go wrong

1. **Variable names.** u10 vs U10 vs 10u vs eastward_wind. ncdump -h first.
2. **Coordinate names.** ERA5 has shipped both time and valid_time across CDS format revisions.
3. **Longitude convention.** 0-360 vs -180-180. If your grid is at -52° and your forcing is at 308°, you may get a silent field of zeros rather than an error. Fix in xarray:

```
ds = ds.assign_coords(longitude=((ds.longitude + 180) % 360) - 180)).sortby("longitude")
```

4. **Coverage.** The forcing must cover the model domain *and* the model time window with a margin. WW3 will not extrapolate off the end of your winds; it will stop, or hold the last field, depending on version.

5.3.2 Verify before you run the model

Always output WND as a model field and look at it. If `ww3_prnc` silently produced nothing useful, a map of the wind WW3 *actually used* shows it in two seconds. This is the cheapest debugging habit in wave modelling.

5.4 Boundary spectra

Locally generated wind sea is only part of the picture. Swell arrives from thousands of kilometres away, and on many coasts it dominates.

```
parent run → ww3_ounp (spectral netCDF at points along the child's boundary)
              ↓
            ww3_bounc → nest.ww3 → read automatically by ww3_shel
```

`ww3_bounc` reads a `spec.list` file naming the spectral netCDF files to use:

```
ls ../configs/GLOBAL_30MIN/SPEC/ww3.*spec.nc > spec.list
ww3_bounc
```

There is no flag in `ww3_shel.nml` to enable boundaries — if `nest.ww3` exists in the run directory it is used. Check `log.ww3`: the input timeline shows boundary updates.

The alternative for global-scale sources: use NOAA's operational GFS-Wave output, or an IFREMER hindcast, as your parent. You don't have to run the global grid yourself.

5.5 Exercise

Take example 02, run it twice — once with boundary spectra, once without — and map the difference in H_s . The result is a map of "how much of the wave climate here comes from somewhere else". For the southern Brazil shelf in winter, it is most of it.

→ [05-nesting.md](#)

Chapter 6

05 — Nesting and multi-grid

Two ways to combine grids of different resolution. They are genuinely different, not two spellings of the same thing.

6.1 One-way nesting: chained `ww3_shel` runs

```
coarse run → ww3_oupn at points along the fine grid's boundary
           → ww3_bounc → nest.ww3
           → fine run
```

Simple, debuggable, embarrassingly parallel across the fine domains, and the standard approach when the parent is somebody else's operational output. The cost: no feedback. The fine grid cannot tell the coarse grid that a storm just generated a lot of energy.

Boundary setup lives in `ww3_grid.nml` for the *fine* grid:

```
&INBND_COUNT_NML
  INBND_COUNT%N_POINT = 2
/
&INBND_POINT_NML
  INBND_POINT(1) = 2  2  F
  INBND_POINT(2) = 2  80 T    ! CONNECT=T fills in everything between
/
```

Rules, both of which catch everyone:

- **Boundary points must be inside the grid.** Never on row/column 1 or N. The propagation stencil needs a cell outside them. This is the most frequently asked question on the WW3 Discussions board.
- `CONNECT = T` fills in every point on the straight line or diagonal from the previous point. Two entries define a whole edge.

Reference configuration: `regtest ww3_tp2.17`.

6.2 Two-way: `ww3_multi`

One executable runs a mosaic of grids simultaneously, exchanging spectra both directions each timestep. Driven by `ww3_multi.nml` instead of `ww3_shel.nml`, with one `mod_def.<grd>` per grid.

The grid definition line carries a rank and a group number. Ranks control which grid wins where they overlap; groups control which grids are computed concurrently. Getting this right is the actual skill in multi-grid WW3, and the reference material is the `mww3_test_01` through `mww3_test_08` regression tests plus the UFS documentation, which describes the `NFGRIDS` / `WINDLINE` / `ICELINE` / `CURRLINE` / `UNIPOINTS` / `WW3GRIDLINE` structure.

`UNIPOINTS` is worth knowing about: it gathers point output from all grids into one unified file, which otherwise is a genuine annoyance.

Two-way is what NOAA runs operationally. It is also substantially harder to debug, because a problem in one grid propagates into all of them.

6.3 Which to use

	One-way	Two-way
Parent is someone else's data	✓ only option	✗
Exploring / learning	✓	
Debugging	✓ much easier	
Storm generated <i>inside</i> the fine domain	✗ feedback lost	✓
Operational forecasting		✓
Many independent coastal domains	✓ trivially parallel	

Start one-way. Move to `ww3_multi` when you have a specific reason.

6.4 Unstructured as an alternative

A single unstructured mesh with 10 km elements offshore and 200 m in the inlet sidesteps nesting entirely. The trade: implicit schemes, PDLIB + ParMetis for parallel decomposition, and mesh generation becomes its own craft (OceanMesh2D, GMSH, SMS). Regtest `ww3_tp2.7` is the reference.

→ [06-output.md](#)

Chapter 7

06 — Output and post-processing

7.1 The seven output types

Configured in `&OUTPUT_TYPE_NML` (what) and `&OUTPUT_DATE_NML` (when) in `ww3_shel.nml`. A stride of '0' disables a type.

1. **Gridded fields** — Hs, periods, direction, etc. on the model grid. → `out_grd.ww3` → `ww3_ounf` → netCDF. The one you'll use 90% of the time.
2. **Point spectra** — full 2D frequency-direction spectra at named locations. → `out_pnt.ww3` → `ww3_ounp` → netCDF. This is the model's actual state, not a summary.
3. **Track output** — fields along a moving track. For satellite collocation. → `ww3_trnc`.
4. **Restart files** — the complete model state, for continuing a run.
5. **Boundary data** — spectra along declared output boundaries, for feeding a child grid. Mostly superseded by using point output + `ww3_bounc` instead.
6. **Separated wave fields** — spatially and temporally coherent wave *systems* tracked across the domain. See the `ww3_syst rk` program and IFREMER's TUTORIAL_WAVETRACK
7. **Coupling fields** — for NUOPC/ESMF/OASIS coupled runs.

7.2 Fields worth knowing

```
DPT CUR WND AST WLW ICE IBG D50 IC1 IC5
HS LM T02 T0M1 T01 FP DIR SPR DP HIG
EF TH1M STH1M TH2M STH2M WN
PHS PTP PLP PDIR PSPP PWS PDP PQP PPE PGW PSW PTM10 PT01 PT02 PEP TWS PNR
UST CHA CGE FAW TAW TWA WCC WCF WCH WCM FWS
SXY TWO BHD FOC TUS USS P2S USF P2L TWI FIC
ABR UBR BED FBB TBB
MSS MSC WL02 AXT AYT AXV
```

(the authoritative list with descriptions is at the top of `$WW3/model/nml/ww3_ounf.nml`)

Ones you'll actually reach for:

Field	What
HS	significant wave height — $4\sqrt{\text{total energy}}$
T01, T02, T0M1	mean periods from different spectral moments. They are not interch
FP, DP	peak frequency and peak direction
DIR, SPR	mean direction and directional spread
EF	the 1D frequency spectrum on the model grid
PHS PTP PDIR PSPR PWS	partitioned : wind sea + N swell systems, separately
USS, TUS	Stokes drift — what you hand to an ocean model
SXY	radiation stresses — what drives nearshore circulation
UST, CHA	friction velocity, Charnock — what you hand back to an atmosphere m
WND, DPT	echo the inputs back. Free, and catches errors instantly.

7.3 Partitioning: the thing to actually understand

A total H_s of 2 m can be a 2 m wind sea, or a 2 m swell, or a 1.4 m sea plus 1.4 m of swell from a completely different direction. These are different oceans and different engineering problems, and a single number cannot distinguish them.

WW3 partitions the 2D spectrum into a wind sea and up to NOSWLL swell systems using a watershed algorithm on the spectral surface, and reports $PHS(n)$, $PTP(n)$, $PDIR(n)$ for each. `FIELD%PARTITION = '0 1 2 3'` in `ww3_ounf.nml` selects which to write (0 = wind sea).

`wavespectra` implements the same family of algorithms (`ptm1 ... ptm5`, plus watershed and wave-age variants) so you can re-partition offline with different parameters without rerunning the model.

7.4 Reading the output

```
import xarray as xr
ds = xr.open_dataset("ww3.nc")
ds.hs.isel(time=-1).plot()
```

For spectra, use `wavespectra` — it has a native WW3 reader and an xarray `.spec` accessor:

```
from wavespectra import read_ww3

dset = read_ww3("ww3.20240701_spec.nc")

dset.spec.hs()          # significant wave height from the spectrum
dset.spec.tp()          # peak period
dset.spec.dpm()         # peak direction
dset.spec.dspr()        # directional spread
dset.spec.stats(["hs", "tp", "dpm", "dspr"])

dset.spec.oned()         # collapse to 1D frequency spectrum
```

```
dset.spec.split(fmin=0.04, fmax=0.10)    # just the swell band
dset.spec.partition.ptm1()               # re-partition offline

dset.isel(time=0, site=0).spec.plot(kind="contourf")  # polar plot
```

A useful consistency check: `dset.spec.hs()` computed from the spectrum should match the HS field `ww3_ounf` wrote. If they disagree, one of them is being computed over a different frequency range than you think.

7.5 Validation

[NOAA-EMC/ww3-tools](#) is the official toolkit: altimeter collocation, NDBC buoy matching, scatter plots, QQ plots, Taylor diagrams, and the standard metric set.

Rough expectations for a regional run with default tuning and decent winds: Hs bias within $\pm 10\%$, scatter index 15–25%. Periods are worse — T_p in particular is a noisy statistic and comparing it point-to-point against a buoy is a good way to feel bad about yourself. Compare T_{0M1} instead, and compare *distributions* as well as time series.

If your first attempt looks dramatically better than that, check that you aren't accidentally comparing the model against itself.

→ [07-physics-choices.md](#)

Chapter 8

07 — Physics choices: which source terms, and why it matters

Everything here is a **compile-time** choice in the switch file. Changing it means a full rebuild. See [../switches/README.md](https://github.com/NOAA-EMC/mosmosis/blob/master/switches/README.md).

8.1 The source-term packages

Package	Origin	Character
ST0	none	Wind input and dissipation off entirely. For pure propagation.
ST1	WAM Cycle 3	Historical. Don't.
ST2	Tolman & Chalikov (1996)	The old WW3 default. Still in use operationally at some sites.
ST3	WAM Cycle 4 / Bidlot	ECMWF lineage.
ST4	Ardhuin et al. (2010)	Saturation-based dissipation plus an explicit swell-dissipation term.
ST6	Rogers, Babanin, Zieger	Built from direct field observations of wind input and wind-wave dissipation.

Pick one. They are **matched sets** — the input and dissipation terms in a package are tuned against each other, and mixing them produces nonsense.

8.2 Why this matters more than you'd like

S_{in} and S_{ds} are each large, each uncertain at the tens-of-percent level, and they nearly cancel. The evolution of the spectrum is the small residual. Two defensible modern packages, run on the same grid with the same winds, can differ by 10–20% in H_s and more in period.

This is not a failure of WW3. It is the honest current state of wind-wave physics. The practical implication: **your model has a tuning, and you should know what it is**. Running ST4 and ST6 on the same case and comparing is one of the most educational things you can do, and `examples/01` is set up for exactly that.

8.3 The nonlinear term

NL1 — the Discrete Interaction Approximation (Hasselmann et al. 1985) — is what almost everyone runs. It approximates the four-wave resonant interaction integral with a single representative quadruplet configuration.

- NL2 computes it (nearly) exactly, at roughly 1000× the cost. Research only.
- NL3 is the Generalized Multiple DIA: several quadruplets with optimised coefficients. Better, more expensive, and the coefficients need fitting — [genes_gmd](#) exists for that.

NL1 is simultaneously the most important and the most approximate term in the model. Some of what ST4/ST6 tuning is compensating for is DIA error, not real physics. Worth knowing when you're tempted to interpret a tuning constant physically.

8.4 Propagation

PR3 UQ — third-order ULTIMATE QUICKEST with the Garden Sprinkler Effect correction — is the default and the right answer for almost everything. Alternatives: PR1 first-order upwind (very diffusive, but rock stable), PR2 second-order, PR0 no propagation.

The **Garden Sprinkler Effect** is worth understanding: with a discrete direction grid, each bin propagates as an independent ray, so a swell field far from its source disintegrates into separate beams. It is a pure discretisation artefact. PR3's correction diffuses across directions to suppress it. More direction bins also helps. If you see striping in a swell field far from the storm, this is why.

8.5 Shallow water

Switch	Physics
BT1	JONSWAP empirical bottom friction. Fine by default.
BT4	SHOWEX movable-bed friction. Needs a D50 sediment map (&SED_NML, &SBT4 SEDMAPD50).
DB1	Battjes-Janssen depth-induced breaking.
MLIM	Miche-style limiter on Hs in shallow water.
TR0/TR1	Triad (three-wave) interactions — surf-zone energy transfer to harmonics.
REF1	Shoreline reflection. Needs a slope map (&SLOPE_NML, &REF1 REFMAP = 2).

If your domain is entirely in deep water, none of this fires and you can leave the defaults.

8.6 Sea ice

IC0 (off) through IC5, plus IS1/IS2 for scattering. IC5 (the extended Fox-Squire model) arrived in v6.06. The ice forcing fields come in through INPUT%FORCING%ICE_CONC and ICE_PARAM1...5 (thickness, floe size, viscosity ...).

Marginal-ice-zone wave physics is an active research area and the parameterisations disagree with each other substantially more than the open-water ones do.

8.7 Tuning namelists

Physics constants live in the file pointed at by GRID%MML (conventionally `namelists.nml`), read by `ww3_grid`:

```
&SIN4
  BETAMAX = 1.52      ! the master gain on wind input. Bump this, everything gets bigger.
/
&SDS4
  SDSBCHOICE = 2
/
&MISC
  FLAGTR = 0
/
```

`ww3_grid` writes the namelists it actually used into `param.scratch` and echoes them to stdout. Check that file — it tells you what the model is really running, as opposed to what you thought you configured. Blocks irrelevant to your switch settings are silently skipped, which means a typo'd namelist name fails quietly.

8.8 A note on tuning

The temptation, once you've validated against a buoy and found a bias, is to reach for BETAMAX. Resist it until you've checked, in this order:

1. Your winds (the dominant error source, usually by a wide margin)
2. Your bathymetry and obstruction grids
3. Your boundary conditions / missing swell
4. Your spectral range — is `FREQ1` low enough for the swell that's actually there?
5. Your resolution
6. *Then* the physics constants

Tuning a source term to compensate for a bad wind field produces a model that's right for the wrong reason and will fail on the next storm.

→ [08-python.md](#)

Chapter 9

08 — Driving WW3 from Python

9.1 The landscape

Tool	Role
pyww3	Generate namelists from typed dataclasses and run the WW3 executables. The
WW3-tools	NOAA's official post-processing and validation: altimeter, buoys, statistics, plo
wavespectra	The xarray library for spectral data. Reading, partitioning, statistics, polar plo
ww3tool	More ambitious wrapper: namelists + run scripts + Slurm submission + plott
bmi-wavewatch3	Download NOAA's <i>published</i> WW3 hindcasts as xarray. Not for running the m
rompy	General pydantic-validated ocean-model configuration. Strong SWAN/SCHISM

9.2 pyww3

The design is simple enough to hold in your head:

- Every WW3 program gets a dataclass — `WW3Grid`, `WW3Prnc`, `WW3Shel`, `WW3Unf`, `WW3Unp`, `WW3Bounc`.
- Every namelist parameter becomes a constructor keyword, with the % flattened to `_`. `SPECTRUM%FREQ1` → `spectrum_freq1`, `TIMESTEPS%DTMAX` → `timesteps_dtmx`.
- Validation happens in `__post_init__`: it checks required files exist and that values are compatible with the namelist definitions.
- All classes inherit `WW3Base`, which provides:
 - `populate_namelist()` — build the namelist text
 - `to_file()` — write it into runpath
 - `run()` — execute the program there, capturing `stdout/stderr`
 - `update_text(block, action=...)` — surgically add or remove a namelist block

That last one exists because the generated namelist contains every block, and `ww3_grid` objects to blocks it doesn't need. Hence the idiom from the docs:

```
W.update_text("&SED_NML", action="remove")
W.update_text("&SLOPE_NML", action="remove")
W.update_text("&CURV_NML", action="remove")
W.update_text("&UNST_NML", action="remove")
```

And the gotcha that follows from it: **after you mutate an attribute, you must regenerate the text.**

```
W.timesteps_dtmax = 1440.
W.text = W.populate_namelist() # <- without this, to_file() writes the OLD value
W.to_file()
```

9.2.1 Minimal end-to-end

```
import datetime
from pyww3.shel import WW3Shel

W = WW3Shel(
    nproc=8,
    runpath="runs/experiment_01/",
    mod_def="runs/experiment_01/mod_def.ww3",
    domain_start=datetime.datetime(2024, 7, 1, 0),
    domain_stop=datetime.datetime(2024, 7, 8, 0),
    input_forcing_winds=True,
    date_field_stride=3600,
    date_point_stride=3600,
    date_restart_stride=3600,
    type_point_file="runs/experiment_01/points.list",
)
W.to_file() # writes ww3_shel.nml
W.run()
print(W.stdout)
```

9.2.2 The honest assessment

The author's own words: *work in progress, API not stable, use at your own risk.* It's a small project with a handful of stars. **Use it anyway, for learning**, because:

- It's a thin layer. You can always print the namelist it generated and read it.
- Being forced to name every parameter in Python teaches you the namelist faster than copying .nml files does.
- Once you understand it, writing your own generator for whatever you actually need is a two-hour job.

What it doesn't do: ASCII-input programs (ww3_outf, ww3_strt, ww3_multi). For those, write the file yourself.

9.2.3 If pyww3 doesn't fit

The namelist format is trivially templatable. Jinja2 plus a YAML config gets you 80% of a wrapper in an afternoon, with the advantage that you control it:

```
from jinja2 import Template
import yaml, subprocess, pathlib

cfg = yaml.safe_load(open("config.yml"))
for prog in ("ww3_grid", "ww3_shel", "ww3_ounf"):
    tpl = Template(pathlib.Path(f"templates/{prog}.nml.j2").read_text())
    pathlib.Path(f"{run}/{prog}.nml").write_text(tpl.render(**cfg))
    subprocess.run([prog], cwd=run, check=True)
```

This is, roughly, what rompy does properly, with pydantic validation instead of hope.

9.3 Work through the exercises

[../exercises/](#) has six graded exercises using pyww3, with solutions. They rebuild examples/01 from Python and then go beyond it into parameter sweeps and spectral analysis.

→ [09-gpu-and-performance.md](#)

Chapter 10

09 — Performance, and the RTX 4090 question

10.1 Your question, answered directly

I would like to try to run the WW3 fortran code on my RTX4090 (nvidia runs fortran, correct?)

Yes, NVIDIA ships a production Fortran compiler. It's nvfortran, part of the free [NVIDIA HPC SDK](#) (descended from the PGI compilers NVIDIA acquired). It supports Fortran 2003 and much of 2008, and offers four separate routes to the GPU:

Route	Flag	What it is
OpenACC	-acc -gpu=cc89	Directive-based. !\$acc parallel loop on existing code. The
OpenMP target	-mp=gpu	Same idea, different standard. !\$omp target teams distribu
CUDA Fortran	-cuda	Full control. attributes(global) kernels, device arrays, exp
do concurrent	-stdpar=gpu	Standard ISO Fortran, no directives at all. The compiler of

Your RTX 4090 is Ada Lovelace, **compute capability 8.9**, so -gpu=cc89. Consumer GeForce cards work fine with nvfortran — you don't need a data-centre card. The SDK bundles CUDA toolkit components but not the driver; nvaccelinfo tells you what driver you have.

But running WW3 itself on it is a different question, and the honest answer is no, not usefully — yet.

10.2 Why WW3 on a GPU is hard

10.2.1 There is no GPU support upstream

NOAA-EMC/WW3 has no GPU code path. No -acc directives, no CUDA kernels, no do concurrent offload. Building it with nvfortran produces a CPU binary.

10.2.2 The one serious attempt got 1.3×

[Ikuyajolu, Van Roekel, Brus, Thomas, Deng & Sreepathi \(2023\), *Geoscientific Model Development* 16, 1445–1462](#) is the reference. Read it before planning anything. Their findings, which you should treat as the baseline expectation:

- They profiled WW3 and found W3SRCMD — the source-term integration — dominates the cost. So that's what they offloaded, with OpenACC, keeping MPI for the rest.
- Tested on Kodiak and Summit (V100 GPUs), meshes of 59K and 228K nodes, 1–32 MPI ranks.
- **Speedup: about 1.3× against 42 CPU cores.** A 35–40% cut in wall time and resource-hours. Real, useful for a climate centre burning millions of node-hours — not the 10× people imagine when they hear “GPU”.
- Packing 3 or 4 MPI ranks per GPU barely changed it.
- **The limiter is host↔device data-transfer bandwidth**, plus W3SRCMD having so many local scalars and arrays that register pressure destroys occupancy.
- Using `!$acc routine` properly would help, but W3SRCMD needs significant refactoring first — it isn't structured for it.
- Their broader point, stated plainly: GPUs need far more parallelism exposed *at one moment* than CPUs, and WW3's structure — global module variables, deep call chains, per-point source-term integration — fights that.
- **This work is not merged upstream.** △ Check whether that's changed since early 2023.

10.2.3 And WW3 is not where the GPU work is going

NOAA has decided the answer is architectural, not incremental. **WAVEWATCH IV** is a ground-up rewrite whose stated drivers include, verbatim, that the rise of GPUs and other advanced architectures requires focusing on memory use and access rather than only on floating-point operations. WW4 Phase IV explicitly targets efficiency on CPUs *and* GPUs, in a code designed for it from the start — new data structures local to each domain, conventional domain decomposition instead of WW3's 2002-era “shuffle”, and C++/Kokkos or Rust instead of Fortran.

Which means any heroic OpenACC work you do on WW3 has a short shelf life. See [10-ww4-and-the-future.md](#).

10.2.4 Your hardware makes it worse, not better

The paper's bottleneck was PCIe/NVLink transfer on Summit nodes, which have **NVLink** between CPU and GPU. You have PCIe 4.0 x16: roughly 32 GB/s, versus NVLink's 50–150 GB/s in that generation. The thing that limited them is *more* limiting for you.

Also worth knowing about the 4090 specifically:

- **FP64 runs at 1/64 of FP32.** Roughly 1.3 TFLOP/s double precision against ~82 TFLOP/s single. That's a deliberate market-segmentation decision, not a defect. WW3 computes mostly in default (4-byte) real, so this may not bite

you directly — but if anything in your build forces `-r8`, or if a library you link does, performance falls off a cliff. Check before you blame the port.

- **24 GB, no ECC.** Fine for a workstation-scale domain; a silent bit flip in a 72-hour hindcast is a real (if unlikely) thing.
- The 4090's actual superpower — enormous FP32/TF32/FP16 tensor throughput — is worthless to WW3, which does none of that.

10.2.5 The structural problem

A WW3 state array is $n_x \times n_y \times n_k \times n_t$. For a modest 81×81 grid with 32 frequencies and 36 directions that's 7.5 million values *per field, per timestep*. The source-term integration touches all of it, per sea point, per timestep, with an adaptive sub-timestep loop (DTMIN) whose trip count varies by point. That's irregular, branchy, register-hungry work over a big array — the opposite of what a GPU wants.

10.3 What to do instead (in order)

10.3.1 1. Use the parallelism WW3 already has

This is the boring answer, and on a workstation it's the right one.

```
# build with switches/switch_lab_mpi (DIST MPI)
mpirun -np $(nproc) ww3_shel
```

WW3's MPI uses "card-deck" domain decomposition over sea points, which balances well. On a modern many-core desktop CPU this scales respectably out to your physical core count. Adding `OMPG OMPH` to the switch enables OpenMP on top (hybrid), useful when MPI decomposition gets too fine.

For unstructured grids with implicit schemes, WW3 has PDLIB + ParMetis decomposition (v6.04+).

Measure this first. You need a CPU baseline before any GPU number means anything.

10.3.2 2. Build WW3 with nvfortran — on the CPU

Worth doing as a stepping stone. It validates the toolchain, and nvfortran's CPU codegen is competitive. The one real obstacle:

Fortran .mod files are compiler-specific. Your distro's `libnetcdf-dev` was built with gfortran and nvfortran cannot read its modules. You must rebuild HDF5 and NetCDF (C and Fortran) with nvfortran first. See [../gpu/build_netcdf_nvfortran.sh](#) for a sketch. The [WRF GPU port repo](#) does the same thing and is a useful cross-reference for the pattern.

Then:

```
export FC=nvfortran CC=nvc CXX=nvc++
export NetCDF_ROOT=$HOME/opt/netcdf-nvhpc
cmake .. -DSWITCH=/path/to/switch_lab_shrd
```

△ Expect to fight it. WW3 is routinely built with gfortran and Intel; nvfortran is less travelled and you may hit Fortran-standards pedantry the other compilers wave through.

10.3.3 3. Measure your own hardware first

Do not take my table of expectations, or anyone else's blog post, on faith. [bench/](#) has three benchmarks:

```
cd bench
make run # WW3-shaped kernel: CPU vs GPU
python3 make_bench_case.py --size medium # then the real model:
bash bench_ww3_cpu.sh $WW3/build case_medium # MPI scaling on your i9
```

kernel_bench runs the same kernel twice — once with the spectrum resident on the device, once copying it in and out each step. The gap between those two numbers is the reason the published WW3 port only reached 1.3×, reproduced on your own PCIe bus in about a minute.

hetero_split answers “can I use the CPU and GPU together?” by sweeping the work split and reporting your optimum. See [bench/README.md](#) for the arithmetic of why the answer is usually “yes, but it gains you almost nothing”.

10.3.4 4. Profile before you port anything

```
nsys profile -o ww3_prof ./ww3_shel # timeline
ncu --set full ./ww3_shel # kernel-level, once you have kernels
```

Or, without NVIDIA tooling at all: build with -pg and use gprof, or just add timers around W3SRCE. Confirm on *your* configuration that source terms dominate. If your case is propagation-heavy (large domain, weak forcing) the hot spot may be somewhere else, and the published profile won't apply.

10.3.5 5. If you still want to port — the realistic plan

Start with the smallest thing that can possibly work:

1. Get [gpu/](#) working first. Confirm `nvfortran -acc -gpu=cc89` produces kernels that actually launch (`NVCOMPILER_ACC_NOTIFY=1` prints every launch).
2. Reproduce the paper's target: W3SRCEMD, OpenACC, one MPI rank per GPU.
3. **Use -acc=multicore as a correctness check.** Same directives, CPU threads instead of a GPU. If the answer changes between multicore and gpu, you have a data-movement bug, not a physics bug.

4. Manage data explicitly. !\$acc enter data copyin(...) once, outside the timestep loop, not per call. The whole game is not moving the spectrum across PCIe every step.
5. Compare against a CPU reference run bit-for-bit where you can, statistically where you can't.

Budget: weeks, not an evening. And the published outcome on better hardware was 1.3×.

10.3.6 6. Point the 4090 at something it's actually good at

Your card is genuinely excellent hardware. Better uses, roughly in order of how much fun they are:

- **gpu/ in this repo.** Toy spectral kernels — DIA-like nonlinear transfer, dispersion solves, propagation stencils — written from scratch for the GPU. These *will* fly, because they're structured for it. You learn OpenACC and CUDA Fortran on problems where the answer is satisfying.
- **Celeris** — a Boussinesq nearshore wave model built GPU-native, interactive, rendering in real time while it solves. If you want to *see* your GPU compute waves tonight, this is the thing. $\triangle$ verify the current repo URL.
- **DualSPHysics** — SPH, written for CUDA from the start. Violent free-surface flows, breaking waves, structure impact.
- **FFT ocean surfaces (Tessendorf).** Take a real directional spectrum out of ww3_ounp, inverse-FFT it into an animated heightfield, render it. Physically it is the *same spectrum* WW3 computes — WW3 gives you its evolution, Tessendorf gives you one realisation of the surface. Wiring the two together is a genuinely good project and it will actually load your GPU.
- **ML wave emulators.** Training a surrogate on WW3 output is a dense tensor workload, which is exactly what the 4090's tensor cores exist for. This is the direction where consumer GPUs are transformative for wave modelling, rather than marginal.

10.4 Quick reference: nvfortran

```
# install: https://developer.nvidia.com/hpc-sdk (tarball or apt repo)
nvaccelinfo                                # driver + device check, prints your compute capability

nvfortran -acc -gpu=cc89 -Minfo=accel prog.f90 # OpenACC, Ada target, tell me what you d
nvfortran -acc=multicore prog.f90             # same directives, CPU threads (correctne
nvfortran -stdpar=gpu -gpu=cc89 prog.f90      # offload `do concurrent`, no directives
nvfortran -cuda -gpu=cc89 prog.cuf            # CUDA Fortran
nvfortran -mp=gpu -gpu=cc89 prog.f90         # OpenMP target offload

export NVCOMPILER_ACC_NOTIFY=1              # print a line per kernel launch
export NVCOMPILER_ACC_TIME=1                 # per-kernel timing summary at exit
```

-Minfo=accel is the one to internalise. It tells you, line by line, which loops were parallelised and — much more importantly — which weren't and why. "Loop not vectorized: data dependency" or "Accelerator restriction: call to procedure with no acc routine" is the compiler telling you exactly what to fix. Most OpenACC work is reading -Minfo output and responding to it.

10.5 Realistic performance expectations, summarised

Approach	Expected on your box
Serial ww3_shel	baseline
MPI over all physical cores	near-linear to your core count. Do this.
MPI + OpenMP hybrid	marginal over pure MPI at this scale
nvfortran CPU build	comparable to gfortran, maybe a little better
Offloading W3SRCEMD to the 4090	Δ optimistically $\sim 1.3\times$ vs a many-core CPU, per the publisher
Custom GPU kernels in gpu/	10–50 $\times$ on the right kernel. Educational, not WW3.
Waiting for WW4	Architected for GPUs. First public release hoped summer 2024

10.6 Sources for everything above

- Ikuyajolu et al. (2023), *GMD* 16, 1445–1462 — <https://gmd.copernicus.org/articles/16/1445/2023/>
- NVIDIA HPC SDK docs (26.5) — <https://docs.nvidia.com/hpc-sdk/>
- OpenACC Getting Started Guide — <https://docs.nvidia.com/hpc-sdk/compilers/openacc-gs/>
- do concurrent offload — <https://developer.nvidia.com/blog/accelerating-fortran-do-concurrent-offload/>
- cc89 = RTX 4090 confirmed in <https://github.com/FahrenheitResearch/wrf-gpu-port> (third-party)
- NCEP Office Note 525, Tolman (2025) — <https://doi.org/10.25923/h7j3-1h25>

→ [10-ww4-and-the-future.md](#), or start playing in [../gpu/](#).

Chapter 11

10 — WAVEWATCH IV: what's coming, and how mature it is

Short version: WW4 is real, it is a full ground-up rewrite in C++ and Rust rather than a new WW3 version, and as of late 2026 it is pre-alpha. Learn WW3. Watch WW4.

11.1 It exists

- Repository: [NOAA-EMC/WW4](#) (v) — “Home of the WAVEWATCH IV™ (WW4™) third-generation wind wave modeling framework”
- Planning document: **NCEP Office Note 525**, *The WAVEWATCH III® Software Modernization Project: Phase I report*, Hendrik L. Tolman, November 2025. [doi:10.25923/h7j3-1h25](#) (v)

Office Note 525 is the single best thing to read if you want to understand where wave modelling is going. It is unusually candid — it publishes the disagreements inside the discussion group rather than papering over them.

11.2 Maturity: a snapshot

Checked against the repository on **2026-09-11** (v):

Signal	Value
Default branch	develop
Commits	36
Releases / tags	none
Stars / forks / watchers	3 / 7 / 4
Open issues / PRs	34 / 1
Top-level dirs	src/ tests/ tools/ templates/ externals/
Build	CMakeLists.txt

Thirty-six commits and no release. This is scaffolding and early core work, not a model you can run. Treat it as a project to follow, not a tool to adopt.

△ Those numbers were true on one day. Check them yourself before quoting them — that's exactly the kind of thing that goes stale fastest.

11.3 Why a new model rather than WW3 v8

The Phase I report lays out the drivers plainly:

- **Parallel concepts are 24 years old.** WW3 uses a “shuffle” method for distributed computing (Tolman 2002a) that load-balances dynamic source-term timestepping beautifully but doesn't scale to exascale. WW4 moves to conventional domain decomposition.
- **Data structures are rooted in the Fortran 90 transition** of nearly two decades ago. WW4 wants spatial data structures local to each domain, for scalability and a smaller memory footprint — and for source-term integration over *areas* rather than individual grid points.
- **Optimisation now means memory, not FLOPs.** The report says this directly: the rise of GPUs and other advanced architectures requires focusing on memory use and access, complementing the traditional focus on floating-point operations. (See lesson 09 — this is the institutional version of the same conclusion.)
- **Fractional stepping fights implicit schemes.** WW3's architecture leans on fractional steps (Yanenko 1971), which works beautifully for explicit propagation and badly for the implicit unstructured-grid solvers everyone now wants.
- **Closer coupling.** UFS wants component models callable at the level of functional units, not wrapped as monoliths. That needs an object-oriented interior.
- **The Fortran compiler pool is shrinking.** Stated as a continuity-of-operations risk: NWS wants to move off Fortran *in a controlled way, before it becomes urgent*.
- **Regression testing has become unsustainable.** Recompiling between individual tests is called out as untenable. WW4 moves to unit and integration testing.

The decision was a **complete bottom-up rewrite in a new repository**, explicitly modelled on how NOAA handled MOM6 (developed separately from MOM4 rather than as an increment). A new repo removes the obligation of backward compatibility and allows “house cleaning” of options that no longer have an owner.

11.4 Languages — the contentious part

The report says outright that language choice was the most contentious question, and that **there is no community consensus**. NOAA/NWS made the call as primary funder:

Language	Role in WW4
C++ (likely with Kokkos)	Initial core language. Conservative, proven, operational centres already using it.
Rust	Named “the modern language of choice for WW4” by NOAA/NWS, for its safety and performance.
Fortran	Solver language only , plus a fast route to an initial operational capability.

Language	Role in WW4
Python	Scripting, workflow, data management, product generation, grid ge
Julia	Considered and declined as a core language by NOAA/NWS: small

The chosen path is the **“dual approach”**: build a C++ core to an operations-ready state in roughly two years, while incrementally building Rust alternatives; a Rust-cored WW4 for operations is a roughly five-year target.

A design principle worth noting, because it's the thing that would let you contribute: WW4 is deliberately keeping a **clean separation between core and solver code**, with language bindings between them — so a contributor writing a new source-term parameterisation can do it in whichever language they're fluent in without learning the core language.

11.5 Yes, it is being written partly by AI agents

The WW4 repository contains an AGENTS.md describing “an agentic AI approach used to create, translate or refactor code using AI agents such as Copilot or Jules, the latter of which has been used extensively in developing the WW4 code from WW3.” (v) The agent is also set up to enforce coding standards, generate doxygen documentation, and write unit tests, all of which are mandated for WW4.

Worth knowing, for two reasons. It's a large, visible, government-operational test of AI-assisted translation of scientific Fortran. And it means the provenance of any given line in WW4 is a live question you should keep in mind when reading it.

11.6 Timeline

From Office Note 525 (v), with the report's own caveat that timelines for a project like this are notoriously difficult:

Phase	What	When
I	Initial choices	complete
II	Language test, governance setup, architecture design	began 1 C
III	Core code development (EMC)	open sour
IV	Initial Model Capability — single domain, CPU and GPU efficiency focus	active co
V	Initial Operational Capability — multi-domain	
VI	Complete first full code	first publ

Those dates are for the C++ path. Rust may take up to five years.

△ We are now past the “summer or fall of 2026” mark for Phase IV. Check the repo's activity, Discussions, and issue tracker to see whether that actually happened.

11.7 What this means for WW3 — and for you

WW3 is not going away soon, but its end is now scheduled. The report is explicit: the cost of maintaining two models long-term will be mitigated by “formally sunseting most support for WW3 once WW4 is mature, with a clearly communicated transition period.” Code with no identified owner willing to port it stays in WW3 and is considered obsolete for WW4.

Practically:

- **Learn WW3 now.** It is the mature, documented, validated, operational model, and it will be for years. Everything in this repo remains the right thing to learn.
- **The concepts transfer completely.** The action balance equation, source-term packages, spectral discretisation, CFL limits, grids, nesting, partitioning — none of that changes. WW4 is a software rewrite, not new physics.
- **The interfaces will not transfer.** Expect namelists to go (the report canvasses ASCII / YAML / namelist and notes the choice follows from the language), expect the binary `mod_def.wv3` / `out_grd.wv3` files to go (consensus to move to NetCDF, with interest in Zarr), and expect the compile-time switch file to be reconsidered — the pushback from researchers about recompiling between runs is recorded in the report.
- **The separate-executables workflow is under review.** `ww3_grid` / `ww3_prep` / `ww3_shel` / `ww3_ounf` as distinct programs is explicitly listed as a design decision to revisit.
- **If you were going to GPU-port WW3 yourself: don't.** WW4 Phase IV targets CPU *and* GPU efficiency in a code architected for it from the start. Any heroic OpenACC work on WW3 now has a short shelf life. See lesson 09.

11.8 One thing that must survive

The report singles out a property of WW3 that WW4 has to keep: **full numerical convergence of its schemes**, achieved through the limiter formulation of Tolman (2002b). Without it you cannot separate numerical error from physical error, which makes the model useless for science. It's a good reminder that “modernisation” has constraints that aren't about software at all.

→ [11-swan.md](#) — the other model you should know.

Chapter 12

11 — SWAN: the other one you should know

If you only ever learn WW3 you will eventually reach a coastal problem where WW3 is the wrong tool and you won't recognise it. SWAN is that tool. It's free, it's a few hours to build, and knowing when to reach for it is a real skill.

12.1 What it is

SWAN — Simulating WAVes Nearshore — is a third-generation spectral wave model from **Delft University of Technology** (Booij, Ris & Holthuijsen). Same governing equation as WW3, same family of source terms, deliberately different numerics.

Current version: **Cycle III version 41.51** (v). Free and open source.

12.2 Where the source actually lives

Three places, and the third is new enough that most tutorials don't mention it:

Source	URL
Official site + downloads	https://swanmodel.sourceforge.io/download/download.htm
SourceForge file releases	https://sourceforge.net/projects/swanmodel/files/swanmodel/
Git — TU Delft GitLab	https://gitlab.tudelft.nl/citg/wavemodels/swan (v)
Release notes / modifications	https://swanmodel.sourceforge.io/modifications/modifications.htm
Implementation manual (build guide)	https://swanmodel.sourceforge.io/download/zip/swanimp.htm

```
git clone https://gitlab.tudelft.nl/citg/wavemodels/swan.git && cd swan
```

Its sibling **SWASH** (non-hydrostatic, phase-resolving, same group) is at <https://swash.sourceforge.io/> and <https://gitlab.tudelft.nl/citg/wavemodels/swash> (v).

Ignore the various GitHub mirrors and vendored copies you'll find. They're snapshots of whatever version someone needed years ago.

12.3 Building it

Modern SWAN builds with **CMake 3.12+**, and the implementation manual recommends **Ninja** as the generator because it's faster than GNU make (v).

```
git clone https://gitlab.tudelft.nl/citg/wavemodels/swan.git && cd swan
cmake -G Ninja -B build -S .
cmake --build build
```

The older workflow (make config then make ser / make omp / make mpi) still exists and is what most older tutorials assume. Either works; CMake is where it's going.

scripts/04_get_swan.sh in this repo does the clone and build.

12.3.1 Switches, SWAN-style

SWAN has the same *idea* as WW3's switch file — compile-time feature selection — but a different mechanism. Options live as **specially-formatted comments inside the .ftn sources**, stripped or activated by a preprocessing step. For example `!/impi` marks MPI code in `swmod1.ftn`, and `!ADC` marks the ADCIRC-coupling hooks scattered across several files (v).

Practical consequence, identical to WW3: if you change what's enabled, you re-build. And the same class of confusion applies — “I turned on MPI and nothing happened” is a stale build directory.

12.4 SWAN versus WW3: the actual difference

Both solve the action balance equation. Both have DIA quadruplets, whitecapping, bottom friction, depth-induced breaking. The difference is **numerics**, and it decides everything else.

	WW3	SWAN
Propagation scheme	Explicit (PR3 UQ, third-order ULTIMATE QUICKEST)	Implicit sweeps
Timestep	CFL-limited. Halve Δx , halve Δt .	No CFL constraint
Sweet spot	Global and basin scale, deep water, long integrations	Coastal, high res
Stationary mode	no	yes — solve direct
Triads (3-wave)	TR0/TR1, limited	Mature; matters
Diffraction	no	approximate (pha
Obstacles	subgrid obstruction grids	explicit obstacle
Ice physics	IC1-IC5, IS1/IS2 — far ahead	minimal
Multi-grid mosaic	ww3_multi, two-way	no
Parallelism	MPI “shuffle” decomposition, OpenMP	OpenMP and MP

The one sentence that matters: WW3's explicit scheme means the timestep shrinks with the grid spacing, so a 50 m coastal grid becomes ruinously expensive; SWAN's implicit scheme doesn't care. That is why the standard architecture in

coastal work is **WW3 offshore** → **SWAN nearshore**, with WW3 spectra as SWAN's boundary condition.

Note the trade: SWAN buys unconditional stability at the cost of numerical diffusion and, in non-stationary mode, accuracy that depends on how many sweeps you let it do. The implementation manual's advice on block-Jacobi versus block-wavefront is precisely about where that trade lands on a parallel machine. WW3's convergence property — the thing Office Note 525 insists WW4 must preserve — is not free, and SWAN made the other choice.

12.5 When to use which

Situation	Reach for
Global or basin-scale hindcast/forecast	WW3
Operational forecasting inside UFS/NOAA	WW3
Sea ice, marginal ice zone	WW3
Bay, estuary, harbour, surf zone at 10–200 m	SWAN
Stationary design wave conditions for engineering	SWAN (stationary mode)
Nearshore circulation via radiation stress	SWAN, or SWAN inside Delft3D
Wave-by-wave runup, overtopping, infragravity	SWASH or XBeach, not either of these
Coupled surge + waves, US practice	ADCIRC+SWAN
Coupled ocean-atmosphere-wave-sediment	COAWST (has both)

12.6 The ecosystem around it

- **Delft3D-WAVE** wraps SWAN and couples it to Delft3D-FLOW. Open source.
- **ADCIRC+SWAN** is the US storm-surge standard; the coupling hooks are in SWAN's source behind the !ADC switch.
- **rompy-swan** (v) — pydantic-validated, type-safe SWAN configuration from Python or YAML, with data interfaces for NetCDF and THREDDS inputs. This is the most polished Python front-end for *any* spectral wave model, and noticeably more mature than anything equivalent for WW3.
- **wavespectra** (v) reads SWAN spectra natively (read_swan), so your post-processing is shared between the two models.
- **swantools** (PyPI) — older, lighter; reads TABLE, SPECOUT and BLOCK output into pandas.
- **OMUSE** packages SWAN for the Oceanographic Multi-purpose Software Environment.

12.7 Exercise

Take examples/02-regional-real-forcing and extend it into a two-model chain:

1. Run WW3 on the 0.1° shelf grid as you already do.
2. Use ww3_ounp to write 2D spectra along a line just offshore of Florianópolis.

3. Build SWAN, set up a ~ 50 m nested grid over the bay, and feed it those spectra as a boundary condition.
4. Compare nearshore H_s against running WW3 alone at the same 50 m resolution.

You will learn three things: that the WW3-only run takes far longer for the same domain (the CFL limit, made personal); that they don't agree; and that converting spectral boundary data between two models' conventions is where the real work in coastal modelling actually goes.

→ Back to [README.md](#).